DevOps, Git & GitHub: Quiz Notes

Quick-revision notes built from your course files: DevOps Introduction, VCS/Git/GitHub, Git Branching Basics, Git Visual Guide

1. What is DevOps?

DevOps = Development + Operations. It is a culture and set of practices where the people who *write* software (Dev) and the people who *run* it on live servers (Ops) work together. Goal: **faster delivery of software with high quality.**

	Dev team	Ops team
Core task	Write code, design features, fix bugs	Deploy to live servers, manage infrastructure, monitor health
Main goal	Speed and innovation	Stability, security, uptime (reliability)
Mindset	Push changes quickly	New changes are a **risk** (bugs, crashes)

- **The problem:** Dev and Ops worked in isolated **silos** (separate boxes): bottlenecks, finger-pointing, late releases. Classic symptom: **"It works on my machine!"**
- **Benefits:** speeds up development, improves collaboration, better quality and faster issue resolution.
- **Supporting tools:** CI/CD pipelines (automatic testing and releasing) and containerization.
- **Without DevOps:** slow releases, silos and miscommunication, manual processes (more bugs and failed deployments), user feedback not integrated quickly.
- **Who uses it:** Netflix, Amazon, Google (frequent updates, high availability).

Key points for the quiz

• Three principles: **Collaboration, Automation, Continuous Improvement.**

• Ops is the team that sees new changes as a risk.

Common mistakes

• DevOps is **not** a tool. It is a culture and a set of practices; tools only support it.

2. VCS, Git and GitHub

VCS (Version Control System): a tool that tracks changes to files. It keeps a full history (what, who, when, why), lets you go back to any older version, and lets many people work together without overwriting. Examples: **Git, Subversion (SVN), Mercurial.**

Git: the most popular VCS. Free, runs on **your computer**, created by **Linus Torvalds** (creator of Linux) in **2005**. **Distributed:** full history stored locally, works offline. Main ideas: commits (saved snapshots), branches (safe space for new features), merge (combine branches).

GitHub: an **online** service that hosts Git repositories: cloud backup and sharing, pull requests and code review, issue tracking, public profiles and open source, GitHub Actions for automation. Alternatives: **GitLab, Bitbucket.**

	VCS	Git	GitHub
What it is	The idea	The tool	The online home
Like	A "word processor"	Microsoft Word	Google Drive + teamwork

Common mistakes

• VCS is the **idea**; Git is **one tool** of that kind.

• Git and GitHub are **not** the same: Git = tool on your computer, GitHub = website that hosts Git projects.

3. The Git Workflow (4 places)

Place	What it is	Analogy
Working Directory	Project folder where you edit code. Changes are **not saved** in Git yet.	Your messy desk
Staging Area	Waiting room for files going into the next commit (**git add**).	Packing a box
Local Repository	Copy on your computer. **git commit** saves a permanent snapshot here.	Sealing the box
Remote Repository	Copy online (GitHub, GitLab, Bitbucket). **git push** uploads here. Also a backup.	Posting the box

```
git add .                  # Working Directory -> Staging Area
git commit -m "message"    # Staging Area -> Local Repo (your computer only)
git push                   # Local Repo -> Remote Repo (GitHub)
```

Key points for the quiz

• Order: **Working Directory -> Staging -> Local Repo -> Remote Repo.**

• Typical workflow: **Edit, Commit, Push, Review** (teammates pull or open a pull request).

Common mistakes

• **git commit** does NOT upload to GitHub. Only **git push** does.

• Forgetting **git add** before **git commit**: nothing is staged, so nothing is committed.

Life of a file: 4 states

State	Meaning
Untracked	Brand-new file that Git does not know about yet.
Tracked	Git is watching it and notices every change.
Staged	Marked and ready for the next commit.
Committed	Saved permanently in project history.

Order: **Untracked -> Tracked -> Staged -> Committed.** A new file always starts **Untracked**. If the folder is not a Git project yet, run **git init** first. Use **git status** to see states.

4. Setup Commands

```
git config --global user.name "Your Name"        # name on your commits
git config --global user.email "you@ext.com"     # email on your commits
git config --global init.defaultBranch main      # default branch name
git config --global --list                       # verify saved settings

git remote add origin https://github.com/User/Repo.git   # link to GitHub
git push -u origin main                                   # first push
```

Key points for the quiz

• **--global** = setting applies to all projects on this computer.

• **origin** = just a nickname for the remote repository URL.

• **-u** = link (track) the local branch to the remote branch, so later you can type plain **git push**.

• Browser login is prompted on the first push.

Common mistakes

• Thinking **origin** is a special Git word. It is only a nickname.

• Saying **-u** means "push to main". It means **link the branches**.

5. Branches and HEAD

Commit = snapshot linked to its parent. **Branch** = a movable label pointing at one commit (creating one is instant, no files are copied). **HEAD** = where you are now; new commits go onto the branch HEAD points at.

```
git branch feature          # create the label (you STAY where you are)
git switch feature          # move HEAD to it
git switch -c feature       # create and switch in one go
git checkout -b feature     # older way to do the same
git switch -                # jump back to previous branch

git branch                  # list local branches (* = current)
git branch -v               # also show last commit of each
git branch -a               # include remote branches
git branch -m old new       # rename
git branch -d feature       # safe delete (only if merged)
git branch -D feature       # force delete (may lose work)
```

Common mistakes

- **git branch name** only creates the branch; it does **not** switch to it. Commits then go to the branch you were already on.
- Deleting an unmerged branch with **-D** can lose work. Prefer **-d**.

6. Merging

	Fast-forward	Three-way merge
When	**main has not moved**; feature is directly ahead	Both branches have new commits (diverged)
New commit	None, the pointer just moves forward	A **merge commit** is created
Git looks at	Two points	Three points: common ancestor and both tips
History shape	Straight line	Diamond

```
git switch main           # go to the branch that RECEIVES the work
git merge feature         # fast-forward if possible, else three-way
git merge --no-ff feature  # always create a merge commit
git merge --ff-only feature  # fast-forward only, otherwise refuse
```

Common mistakes

- Running **git merge** while on the wrong branch. You merge **into** the branch you are on.

7. Merge Conflicts

A **conflict** happens when both branches changed the **same line of the same file**. Git cannot choose, so it stops and asks you. It is normal, not a sign that something is broken.

```
git merge feature     # CONFLICT
git status            # look for "both modified" files
# open the file, remove <<<<<<<  =======  >>>>>>>, write the final line
git add index.html    # tell Git it is resolved
git commit            # or: git merge --continue

git merge --abort     # panic button: back to before the merge
git rebase --continue / git rebase --abort   # during a rebase
```

Common mistakes

- Leaving the conflict markers inside the file.
- Forgetting **git add** after fixing the file.

8. Rebase, Squash, Linear and Clean History

Linear history: git log --graph shows one straight line (no diamonds). **Clean history:** every commit has a clear purpose (no "wip" or "fix typo" commits).

Rebase: replays your branch's commits on top of a new base. Commits stay separate, but their IDs change (f1 becomes f1').

Squash: combines several commits into **one**.

```
git checkout feature
git rebase main                 # replay feature on top of main
git checkout main
git merge feature               # now a plain fast-forward

git merge --squash feature && git commit -m "Add login feature"  # way 1
git rebase -i HEAD~4              # way 2 (pick / squash)

git config --global pull.rebase true   # rebase instead of merging on pull
```

Key points for the quiz

- **Rebase keeps every commit separate; squash makes them all one.**
- **Golden rule:** never rebase or squash a branch that is already pushed and used by others.
- If you must, use **git push --force-with-lease**, never plain **--force** (it will not overwrite work you have not seen).

9. Git Flow

Git Flow is a workflow where every kind of work lives on a branch of a fixed type.

Branch	Created from	Merged into	Naming
main	n/a	n/a	Stable, tagged code only
develop	main	main (through a release)	One long-lived branch
feature/*	develop	develop	feature/login, feature/PAY-42-checkout
release/*	develop	main **and** develop	release/1.1.0
hotfix/*	main	main **and** develop	hotfix/payment-crash

```
git checkout -b develop main
git checkout -b feature/login develop
git checkout develop && git merge --no-ff feature/login
git checkout -b release/1.1 develop
git checkout main && git merge --no-ff release/1.1 && git tag v1.1
git checkout -b hotfix/crash main
```

Key points for the quiz

- Naming rules: **lowercase**, separate words with **-**, add the **ticket number** if you have one.
- **release** and **hotfix** merge into **both** main and develop.

Common mistakes

- Mixing up starting points: **feature** starts from **develop**; **hotfix** starts from **main** (it fixes live code).

10. Remotes, Stash and Detached HEAD

```
git clone <url>                 # copy a repo from the server
git push -u origin feature      # publish a branch and link it (once)
git fetch origin                # download news, change nothing of yours
git pull                        # fetch + merge
git pull --rebase               # fetch + rebase (straighter history)

git stash                       # shelve uncommitted work
git stash pop                   # bring it back
git stash list                  # see all stashes
git switch --detach c2          # look at an old commit (detached HEAD)
git switch -c experiment        # save work made while detached
```

Key points for the quiz

• **fetch** is safe (download only). **pull = fetch + merge** (it changes your branch).

• **Detached HEAD** = HEAD points at a commit, not a branch. Create a branch to keep any work done there.

Common mistakes

• Treating fetch and pull as the same thing.

• Committing in detached HEAD without creating a branch first (the commits are easy to lose).

Last-minute Revision Sheet

Question	Answer
DevOps stands for?	Development + Operations
3 DevOps principles?	Collaboration, Automation, Continuous Improvement
Who created Git, and when?	Linus Torvalds, 2005
Git vs GitHub?	Git = tool on your computer; GitHub = online hosting
Which command uploads to GitHub?	git push (commit only saves locally)
4 file states?	Untracked, Tracked, Staged, Committed
What does -u do?	Links local branch to remote branch
Create and switch to a branch?	git switch -c name (or git checkout -b name)
Fast-forward vs three-way?	No merge commit (main did not move) vs merge commit (both moved)
When does a conflict happen?	Same line of the same file changed in both branches
Undo a messy merge?	git merge --abort
Rebase vs squash?	Rebase keeps commits separate; squash makes them one
Golden rule?	Never rebase or squash a pushed, shared branch
Hotfix starts from / merges into?	main / main and develop
fetch vs pull?	fetch downloads only; pull = fetch + merge
-d vs -D?	-d safe delete (merged only); -D force delete (may lose work)